AI PRACTICE - Flask Login/Register Example

This project is a simple Flask-based login and registration system using SQLite for storage. It demonstrates how to build a web

What this project does

- Provides a registration form for new users.
- Stores users in an SQLite database (`data.sqlite3`).
- Hashes passwords before saving them using `werkzeug.security.generate_password_hash`.
- Validates login credentials using `werkzeug.security.check_password_hash`.
- Uses HTML templates for pages: home, login, register, and success.
- Provides a homepage button to download the generated PDF documentation.
- Logs important app behavior through the database and user interface.

What we did step by step

1. Created `flask_app.py` as the main Flask application.
2. Set up `templates/` with `index.html`, `login.html`, `register.html`, and `success.html`.
3. Added a SQLite database table in `init_db()` to store users.
4. Built `/register` and `/login` routes.
5. Used `generate_password_hash()` to store hashed passwords safely.
6. Updated login logic to verify passwords with `check_password_hash()`.
7. Converted existing plain-text passwords in `data.sqlite3` to hashed values.
8. Removed the temporary helper script used for conversion.
9. Added a homepage button and Flask route to download `project_documentation.pdf`.
10. Verified the app and database, ensuring all stored passwords are hashed.
11. Pushed the project to GitHub and resolved any merge conflicts.

Project structure

- `flask_app.py` — main application logic.
- `requirements.txt` — required Python packages.
- `data.sqlite3` — SQLite database file.
- `logi.txt` — optional plaintext append log from older compatibility code.
- `templates/` — HTML templates used by Flask.

How the programming works

`flask_app.py`

- Imports:
- `Flask`, `render_template`, `request`, `send_from_directory` from Flask for web routing, templates, and serving files.
- `os`, `sqlite3`, `datetime` from Python standard library.
- `generate_password_hash`, `check_password_hash` from `werkzeug.security` for secure password handling.

- `DB_FILENAME`:
- Sets the SQLite file path next to `flask_app.py` using `app.root_path`.

- `init_db()`:
- Creates the `users` table if it does not exist.
- The table has fields: `id`, `full_name`, `email`, `username`, `password`, `created_at`.
- It ensures `email` is unique and `password` is stored as text.

- `@app.route('/')`:
- Renders `index.html` as the home page.

- `@app.route('/login', methods=['GET', 'POST'])`:
- GET: shows the login form.

- POST: reads `email` and `password` from the form.
- Queries the user row by email.
- Verifies the provided password with `check_password_hash()`.
- Shows `success.html` on success, otherwise returns the login form with an error.

- `@app.route('/register', methods=['GET', 'POST'])`:
- GET: shows the registration form.
- POST: reads `full_name`, `email`, and `password`.
- Hashes the password with `generate_password_hash()`.
- Inserts the user into the database with the current timestamp.
- Shows `success.html` after successful registration.

- `@app.route('/download-documentation')`:
- Serves the generated PDF file `project_documentation.pdf` from the project root.
- Returns the file as an attachment so users can download it from the homepage.

- `if __name__ == '__main__':`
- Calls `init_db()` again and runs the Flask development server with `debug=True`.

Key functions and libraries

Flask

- `Flask(__name__)` — create the app.
- `@app.route()` — define URL routes.
- `render_template()` — render an HTML template.
- `request.form.get()` — retrieve submitted form data.

SQLite (`sqlite3`)

- `sqlite3.connect(DB_FILENAME)` — open the database.
- `conn.cursor()` — create a cursor for SQL operations.
- `cursor.execute()` — run SQL statements.
- `cursor.fetchone()` — fetch a single query result.
- `conn.commit()` — save changes.
- `conn.close()` — close the connection.

Werkzeug security

- `generate_password_hash(password)` — hash a plain-text password.
- `check_password_hash(hashed_password, password)` — verify a password against its hash.

Templates

- The app uses HTML templates in `templates/` for each page.
- `register.html` collects full name, email, and password.
- `login.html` collects email and password.
- `success.html` shows a success message after login or registration.

Setup and running the project

1. Open a terminal in `C:\Users\TARIQ NASHEED\OneDrive\Desktop\AI PRACTICE`.
2. Create and activate a virtual environment:

```powershell
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

3. Install dependencies:

```powershell
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

4. Start the app:

```powershell
.\.venv\Scripts\python.exe flask_app.py
```

5. Open `http://127.0.0.1:5000/` in your browser.

Security notes

- Passwords are hashed before storing in the database.
- Never store plain-text passwords in production.
- The app is for learning and practice only.
- For production, add HTTPS, input validation, stronger session management, and CSRF protection.

GitHub and repository notes

- This project has been pushed to a GitHub repository.
- Conflicts were resolved manually in `README.md`.
- `README.md` now contains complete documentation and project history.

PDF Documentation

A PDF version of this documentation is also generated in the project root as `project_documentation.pdf`.

REST APIs

What is a REST API?
- In simple terms, a REST API is an architectural style used to create separation of concerns between the client (user end) and
- REST stands for Representational State Transfer.
- It is an architectural style for designing networked applications.
- A REST API exposes resources (data entities like users, products, posts) via URLs and standard HTTP methods.

Key principles
- Resources: Everything is a resource identified by a URI, e.g. `/users/123`.
- Stateless: Each request contains all information needed; the server does not store client session state.
- Uniform interface: Use standard HTTP methods consistently:
- `GET` = retrieve
- `POST` = create
- `PUT` = update/replace
- `PATCH` = partial update
- `DELETE` = delete
- Representation: Resources are exchanged as representations, typically JSON or XML.
- Client-server separation: Client and server are separate roles; the client handles UI, the server handles data and logic.
- Layered system: Clients don’t need to know if they talk to the actual server, a proxy, or a load balancer.

Why REST is useful
- Simple and widely adopted
- Works over HTTP without special protocols
- Easy to consume from many programming languages

- Clean separation between client and server
- Scales well for web and mobile applications

HTTP Protocol

What is HTTP?
- HTTP is the Hypertext Transfer Protocol.
- It is the foundation of data communication for the World Wide Web.
- It defines how clients and servers exchange messages.

HTTP request structure
- Request line: method + path + protocol version
Example: `GET /users/123 HTTP/1.1`
- Headers: metadata such as `Host`, `Content-Type`, `Accept`, `Authorization`
- Body: optional data for methods like `POST`, `PUT`, or `PATCH`

HTTP methods
- `GET`: read resource
- `POST`: create resource
- `PUT`: update/replace resource
- `PATCH`: partial update
- `DELETE`: remove resource
- `HEAD`: request headers only
- `OPTIONS`: discover supported methods

HTTP responses
- Status line: protocol + status code + reason phrase
Example: `HTTP/1.1 200 OK`
- Headers: metadata like `Content-Type`, `Cache-Control`, `Location`
- Body: data payload, often JSON for APIs

Common HTTP status codes
- `200 OK`: successful read or action
- `201 Created`: resource created
- `204 No Content`: success with no response body
- `400 Bad Request`: invalid request
- `401 Unauthorized`: missing or invalid credentials
- `403 Forbidden`: no permission
- `404 Not Found`: resource not found
- `500 Internal Server Error`: server failure

Why HTTP matters
- It is universal for web communication.
- REST APIs use HTTP methods and status codes to express intent clearly.
- It is supported by browsers, mobile apps, servers, libraries, and command-line tools.

---

cURL

What is cURL?
- cURL is a command-line tool for transferring data with URLs.
- It supports HTTP, HTTPS, FTP, and many other protocols.
- It is widely used for testing APIs and making requests from scripts.

Why use cURL?

- Fast way to test REST endpoints
- Works without writing code
- Useful for debugging headers, payloads, authentication, and response status
- Common in documentation examples and CI scripts

Example usage
```bash
curl https://api.example.com/users/123
```

POST with JSON:
```bash
curl -X POST https://api.example.com/users \
-H "Content-Type: application/json" \
-d '{"name":"Alice","email":"alice@example.com"}'
```

GET with authorization:
```bash
curl -H "Authorization: Bearer TOKEN" https://api.example.com/profile
```

Why cURL is important
- It lets developers verify server behavior directly.
- It helps isolate whether a problem is in the client code or API server.
- It is the “universal client” for HTTP-based services.

JSON

What is JSON?
- JSON stands for JavaScript Object Notation.
- It is a lightweight, text-based data interchange format.
- It is human-readable and easy for machines to parse.

JSON structure
- Objects: `{ "name": "Alice", "age": 30 }`
- Arrays: `[1, 2, 3]`
- Values: strings, numbers, booleans, null, objects, arrays

Example
```json
{
"id": 123,
"name": "Alice",
"email": "alice@example.com",
"roles": ["admin", "editor"]
}
```

Why JSON is used
- Simple syntax
- Language-agnostic support
- Ideal for REST APIs
- Commonly used for request and response payloads
- Easier to read and debug than XML

Caching

What is caching?
- Caching stores copies of data or responses so that future requests can be served faster.
- It reduces repeated work, lowers latency, and decreases server load.

Where caching happens
- Client cache: browser or app stores responses locally
- Proxy cache: intermediate server caches responses
- Server cache: application server stores computed results
- Database cache: query results stored in memory or cache systems like Redis

HTTP caching headers
- `Cache-Control`: specifies caching policies
- `Cache-Control: max-age=3600` means cache for one hour
- `Cache-Control: no-cache` means validate before reuse
- `Expires`: defines absolute expiration time
- `ETag`: unique tag for a resource version
- `Last-Modified`: last update timestamp
- `If-None-Match` / `If-Modified-Since`: conditional requests to validate cache

Why caching is used
- Improves performance by avoiding repeated computation or database reads
- Reduces bandwidth usage
- Makes APIs faster and more responsive
- Helps systems scale under load
- Allows content to be reused safely when it has not changed

How these pieces fit together

- HTTP is the protocol used to send and receive REST API requests.
- REST APIs use HTTP methods and URLs to expose data and operations.
- JSON is the most common format for encoding resource requests and responses.
- cURL is a tool for sending HTTP requests and testing REST APIs.
- Caching makes repeated API requests faster and reduces server work.

Example flow
1. A mobile app sends a `GET /api/users/123` request over HTTP.
2. The server returns a JSON response with user data.
3. The client may cache that response locally for later use.
4. The next request can reuse the cached response if data is still fresh.
5. If the client wants to create a new user, it sends a `POST` request with JSON body.
6. The server processes the request and returns a status code like `201 Created`.

Deep explanation: why this matters

For developers
- REST + HTTP + JSON create a standard way for apps to communicate.
- It makes APIs easier to design, document, and consume.
- Tools like cURL let you test APIs independently of UI or client code.

For performance

- Caching prevents repetitive work and improves speed.
- HTTP caching lets clients and intermediaries reuse responses safely.
- Proper caching design is essential for scalable web applications.

For compatibility
- HTTP is universal, so any platform can call REST APIs.
- JSON is supported by virtually every programming language.
- This combination enables web, mobile, desktop, and IoT systems to interact.

For maintainability
- REST APIs keep resource behavior predictable.
- HTTP status codes make error handling standardized.
- Clear request/response formats reduce bugs and simplify integration.